Internship Notebook

Deep Learning for Image Classification and Medical Image Segmentation

Todoruț Aurel-Dan

July – August 2026

1 Header

Group	30434
Student name and surname	Todoruț Aurel-Dan
Faculty	Faculty of Automation and Computer Science, Technical University of Cluj-Napoca
Specialization	Computer Science, year 3
Place of the internship	Faculty of Automation and Computer Science, Str. George Barițiu no. 28, room 37, Cluj-Napoca
Internship period	13.07.2026 – 21.08.2026 (6 weeks)
Total number of hours	200 hours
Tutor	Conf.dr.ing. Tiberiu Marita tiberiu.marita@cs.utcluj.ro
Student contact	todorut.dan@yahoo.com

2 Summary

During this internship I worked on 2 deep learning projects in PyTorch. The first was image classification on CIFAR-10, where I built a convolutional neural network from scratch and ran it on three tasks: cat versus dog (2 classes), only the animal classes (6), and the full dataset (10). The best test accuracy was 88.25% on the binary task and 88.85% on the ten-class task. I also ran ten experiments changing one setting at a time, to see which parts of the network actually matter.

The second and larger project was medical image segmentation on the LiTS dataset (Liver Tumor Segmentation). Here the goal is to give every pixel of a CT slice one of three labels: background, liver or tumour. I implemented two architectures, a standard U-Net and an Attention U-Net, and compared them under identical conditions. The final foreground Dice scores were 0.8222 for U-Net and 0.8247 for Attention U-Net, so the attention gates did not give a real improvement.

The first week was mostly learning, using the Stanford course material. After that I spent about two weeks on classification and three weeks on segmentation.

What I did not expect is how much of the work is checking that results mean what they seem to mean. I found that the first version of the segmentation dataset had label files damaged by JPEG compression, that my first data split let almost identical images appear in both training and test sets, and that pixel accuracy looks good while the model finds nothing. Every one of those problems was silent (no error message). I ended up writing small tools whose only job is to verify my own work, and they caught several mistakes.

3 Detailed description of the activities

3.1 Weekly diary

Week 1 (13–17 July): setup and learning

The first days went into preparing the environment and studying, because I had only used neural networks in one course before and never on images of this size.

Environment. I installed Miniconda and made a separate conda environment instead of installing packages into the system Python. It turned out to be very useful: later I broke a PyTorch installation and, because everything was inside one environment, fixing it was a matter of recreating that environment instead of repairing the whole system. I installed PyTorch with CUDA support and checked that `torch.cuda.is_available()` returned `True`, so training would run on my laptop GPU (NVIDIA RTX 3050) instead of the CPU. I used PyCharm as editor.

Learning. I went through the parts of Stanford ("Convolutional Neural Networks for Visual Recognition") that I needed, mainly the lectures on convolution and pooling, on training neural networks, and the notes on regularization. The ideas I had to understand before writing code were:

- what a convolution filter computes, and why weight sharing lets one filter find the same pattern anywhere in the image;
- the receptive field, and why two stacked 3×3 convolutions see a 5×5 area but use fewer weights than one 5×5 filter;
- what max pooling does, and what information it throws away;
- batch normalization, dropout and data augmentation as three different ways of fighting overfitting.

I also read about segmentation, which was completely new to me. The main change of thinking is that the output is not one label per image but one label per pixel, so the network has to produce something the same size as its input. That is where the (encoder -> decoder) shape comes from.

Week 2 (20–24 July): classification, implementation

This week I built the classification project. I split the code into separate files from the start, because I would want to run many experiments: `config.py` (one object holding every setting), `data.py`, `model.py`, `train.py`, `evaluate.py` and `gradcam.py`.

Data. CIFAR-10 has a train and a test split but no validation split, so I cut a validation set out of the training data. This matters because if you tune settings by looking at the test set, you are effectively training on it. Augmentation was random horizontal flips and small random crops, applied only to the training set and never to validation or test.

Model. The network is made of repeated blocks. Each block has one or more 3×3 convolutions, each followed by batch normalization and ReLU, then a max pooling layer that halves the resolution. The number of filters doubles every block. After the last block a global average pooling layer flattens everything, then one hidden fully connected layer, then the output layer with one score per class. Everything is configurable, so I could change the depth or the width from the command line without editing the model. The default network has **320,994** trainable parameters.

Three tasks from one codebase. I made the set of classes a setting instead of hard coding it, so the same code runs on cat versus dog, on the six animal classes, and on all ten classes.

Problems. Two things cost me time. On Windows the data loader crashed with a memory error when I used several worker processes, each worker imports PyTorch and Windows backs the shared memory with the paging file, which was too small. I ran with zero workers instead.

And my first runs were much slower than they should have been until I realised the workers were being recreated at every epoch.

Week 3 (27–31 July): classification experiments

With the pipeline working, this week was about running experiments and understanding results rather than writing new features.

Metrics. I implemented accuracy, balanced accuracy, macro F1, ROC-AUC and a confusion matrix. The confusion matrix was the most useful, because it shows which classes get mixed up and not only how often.

Baselines. Table 1 lists the three tasks.

Task	Classes	Epochs	Best val acc	Test acc	Macro F1
Cat vs dog (binary)	2	40	0.8850	0.8825	0.8825
Animals only	6	50	0.8713	0.8680	0.8680
Full CIFAR-10	10	50	0.8924	0.8885	0.8881

Table 1: Baseline results for the three classification tasks.

Something surprising here: the ten class task scores *higher* than the six class one, even though it has more classes. The reason is that CIFAR-10 contains four vehicle classes (airplane, automobile, ship, truck) which are easy to separate from animals, while the six class task keeps only animals, which look much more alike. More classes does not automatically mean a harder problem.

On the binary task the confusion matrix was $[[896, 104], [131, 869]]$: cat recall 0.896, dog recall 0.869, so the errors were roughly balanced.

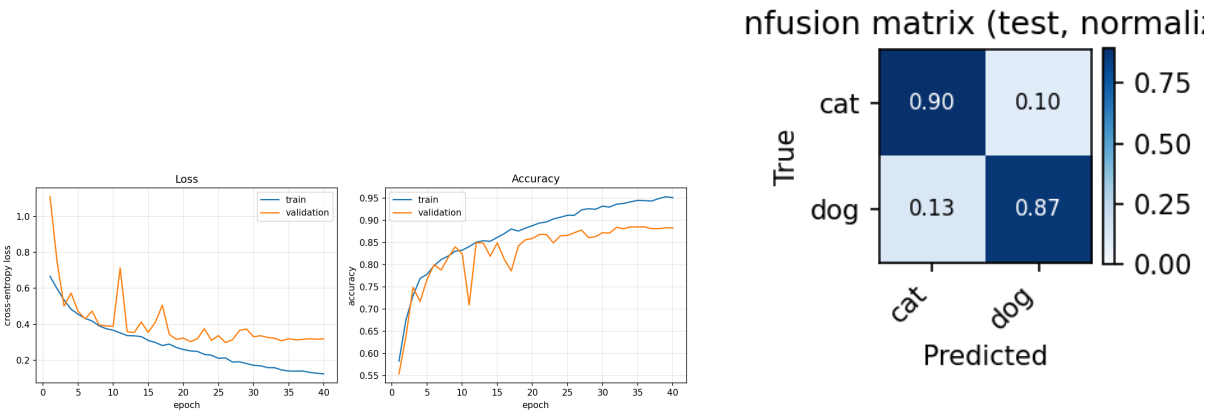


Figure 1: Binary task: training curves over 40 epochs (left) and the test confusion matrix (right).

Experiments. I ran ten variants, each changing one thing, all for 20 epochs so they were comparable (Table 2). Because the budget is shorter, the baseline here scores 0.8575 instead of the 0.8825 from the 40-epoch run, the point of the table is the differences between rows, not the absolute values.

Variant	What changed	Params	Test acc	vs baseline
baseline	3 blocks, 32 filters, dropout 0.3, aug on	320,994	0.8575	–
no_dropout	dropout 0 instead of 0.3	320,994	**0.8720**	+0.0145
wide_64	64 base filters	1,212,610	0.8695	+0.0120
deep_4blocks	4 conv blocks instead of 3	1,239,522	0.8630	+0.0055
label_smooth	label smoothing 0.1	320,994	0.8580	+0.0005
convdrop	spatial dropout 0.1 per block	320,994	0.8550	−0.0025
sgd	SGD + momentum instead of Adam	320,994	0.8440	−0.0135
no_augment	no random crop or flip	320,994	0.8410	−0.0165
shallow_2blocks	2 conv blocks instead of 3	82,914	0.8130	−0.0445
no_batchnorm	BatchNorm removed	320,546	**0.7735**	**−0.0840**

Table 2: Hyperparameter experiments on the binary task, 20 epochs each.

Three things I learned from this table:

- **BatchNorm matters most by far.** Removing it costs 8.4 percentage points, more than every other change put together. It changes almost no parameters (320,546 versus 320,994), so this is not about capacity.
- **Dropout was actually hurting.** Turning it off improved test accuracy by 1.5 points. With augmentation already on, and a network this small, dropout was too much regularization.
- **Depth matters more than width.** Going from 3 blocks to 2 costs 4.5 points while removing 74% of the parameters, but going from 32 to 64 filters buys only 1.2 points for four times the parameters.

Overfitting test. The experiment was a run designed to overfit: no augmentation and no dropout. The gap between training and validation accuracy grew to about 15 points and the validation loss started going up while training loss kept falling. That is the picture of a model memorising its training data.

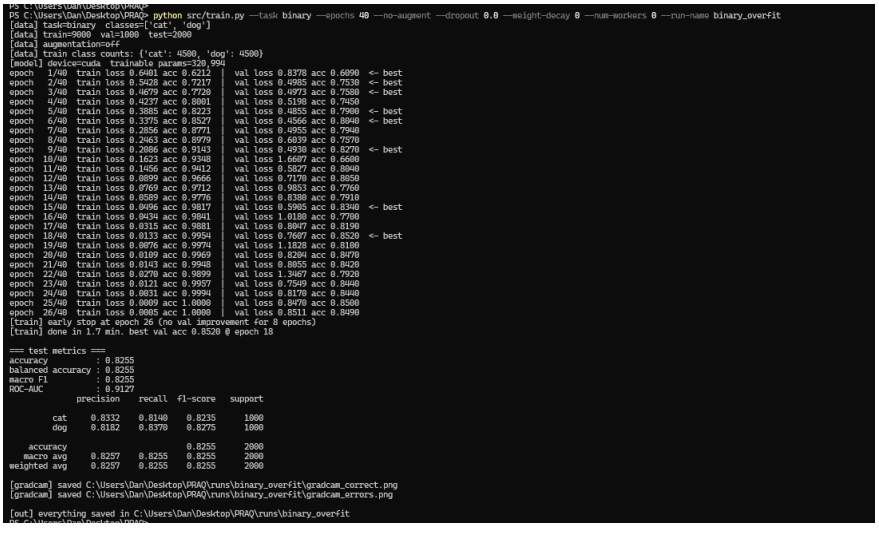


Figure 2: The overfitting run: no augmentation and no dropout. Training accuracy keeps climbing while validation accuracy stops improving, and the validation loss turns upward. This is the signal to stop training or add regularization.

Grad-CAM. I implemented Grad-CAM to see which part of an image the network was using. It takes the gradient of the predicted class with respect to the last convolutional layer and turns it into a heat map over the input. I produced one figure for correct predictions and

one for mistakes. On the correct ones the heat sits on the animal; on several mistakes it sits on the background, which explains the error.



Figure 3: Grad-CAM on correct predictions (left) and on mistakes (right).

Week 4 (3–7 August): segmentation, learning and dataset study

The second project was segmentation of the liver and liver tumours in CT slices. This was harder in every way, so the first part of the week was reading again.

Learning. I studied the U-Net architecture and skip connections, then the Attention U-Net idea of filtering those skip connections with attention gates. I also had to learn some CT basics: what an axial slice is, what Hounsfield Units are, and why the liver only appears in part of a scan.

Dataset study. Instead of coding immediately, I wrote a program that only reads the dataset and describes it. This was the best decision of the whole internship, because it found real problems:

- **131 patients**, **58,638** slices of 256×256 pixels, with between 74 and 987 slices per patient.
- **67.4% of slices contain no liver at all**, because a scan covers more of the body than the liver. Training on them mostly teaches the network to answer "background".
- The classes are extremely unbalanced: background 97.74%, liver 2.15%, tumour **0.12%**. Background is about 840 times more common than tumour.
- The first version of the dataset I downloaded stored the masks as **JPEG**. JPEG is lossy, which is fine for photographs but wrong for label images, where the pixel value *is* the class number. I found pixels with value 3 in a dataset that only has classes 0, 1 and 2, which proves the labels had been changed by compression.

Because of that last point I switched to a **PNG** version of the same dataset, which is lossless. In that version the two label classes are stored as two separate binary masks per slice, one for the liver and one for the lesion, which have to be merged into a single three class mask. The merge order matters: the liver mask also covers the lesion, so the lesion must be written last. Otherwise every tumour pixel is overwritten and the class disappears completely, with no error message.

Week 5 (10–14 August): segmentation implementation

Data pipeline. The important detail is how the data is split. My first version split slices randomly, which is wrong: slices 240 and 241 of the same patient are a few millimetres apart and look almost identical, so a random split puts nearly the same image in both training and test.

The scores look good but they measure memorisation. I changed it to split **whole patients**: the 131 patients are shuffled and divided, and every slice follows its patient. The scores dropped and became honest. The final split is 91 / 20 / 20 patients, giving 13,745 / 3,151 / 2,247 slices.

Augmentation needed care too. The image and its mask must get exactly the same random transformation, or the labels stop matching the picture. I used only flips and 90-degree rotations, because those keep mask values as exact integers, and masks are always resized with nearest neighbour interpolation, never bilinear, interpolating between class 1 and class 2 would invent a class 1.5.

Models. I built both architectures from the same blocks so the comparison would be fair: `DoubleConv` (two 3×3 convolutions with batch normalization and ReLU), `Down` (max pooling then `DoubleConv`), `Up` (transposed convolution, concatenate the skip, then `DoubleConv`) and `AttentionGate`, used only in the Attention U-Net.

Loss. Because of the imbalance I used Dice loss combined with cross-entropy. Dice is computed per class and divided by that class's own size, so tumour counts as much as background even though it is 840 times smaller. Plain cross-entropy on its own would be dominated by background.

Metrics. I computed a confusion matrix and then Dice, IoU, precision and recall per class. I did not use pixel accuracy as the main number: a model predicting "background" everywhere scores about 97% accuracy and segments nothing. The number I used to pick the best model is the **mean foreground Dice**, averaging only liver and tumour.

Week 6 (17–21 August): training, comparison, documentation

First runs. I trained on my laptop GPU first. One epoch took about 6 minutes, so 40 epochs took just under 4 hours. While it ran I watched `nvidia-smi` and saw GPU utilisation of only about 68% with temperature at 84–86 °C, meaning the GPU was partly waiting for data and also thermally limited.

A better GPU. Workstation with an RTX 2070, which I used over remote desktop. I set up the same conda environment there, copied the code, and downloaded the dataset again locally instead of transferring 3.4 GB over the remote connection. On Linux the data loader could use several worker processes – the thing that crashed on Windows, and utilisation went to **98%** with temperature down to 66 °C. I ran training inside `tmux` so the job would survive the remote connection dropping, which it did more than once.

Documentation. Finally I generated the architecture diagrams and parameter tables automatically from the code, so they cannot go out of date, and wrote a tool that produces the comparison table and figures.

One more thing at the end of the week. I wrote a 3D viewer so I could look at a whole patient at once instead of one slice at a time, described at the end of the next section. It showed me something the per-slice numbers had not: the predicted liver comes out in 23 disconnected pieces, which is not a thing a Dice score can tell you.

3.2 Project description: liver and tumour segmentation

This section describes the larger project in detail. My personal contribution is the whole implementation, data pipeline, both architectures, loss, metrics, training loop, evaluation and checking tools. I did not use an existing segmentation framework.

Problem

Given one axial CT slice of the abdomen, label every pixel as background, liver or tumour. The motivation is that radiologists currently outline the liver and every lesion by hand across hundreds of slices per patient, which is slow and differs between readers.

Dataset

LiTS (Liver Tumor Segmentation Benchmark), the version exported as 2D PNG slices. Table 3 lists the properties that changed my design.

Property	Value	What it forced
Patients	131	split by patient, not by slice
Slices	58,638 at 256×256	
Slices with no liver	67.4%	dropped from training
Usable slices	19,143	
Background pixels	97.74%	accuracy is meaningless
Liver pixels	2.15%	
Tumour pixels	0.12%	Dice based loss needed
Masks	2 binary PNGs per slice	merged, lesion written last

Table 3: Dataset properties and the design decision each one forced.

Architectures

Both networks share the same encoder and decoder. The encoder halves the resolution four times while doubling the channels ($32 \to 64 \to 128 \to 256 \to 512$), so a 256×256 input becomes 16×16 at the bottleneck. The decoder does the reverse, and at every level it concatenates a skip connection carrying the high resolution detail that pooling threw away.

The only difference is what happens to those skips:

- **U-Net**: the skip is concatenated exactly as the encoder produced it, so everything crosses over, kidneys, spine, bowel and all.
- **Attention U-Net**: the skip first goes through an attention gate. The gate takes the coarser tensor from the level below as a "gating signal", combines it with the skip using 1×1 convolutions, and produces a weight $\alpha \in (0, 1)$ per pixel. The skip is multiplied by α before being concatenated, so regions the decoder thinks are irrelevant get suppressed:

$$\alpha = \sigma(\psi(\mathrm{ReLU}(W_g\, g + W_x\, x))), \qquad \text{output} = x \odot \alpha.$$

The parameter counts are **7,762,531** for U-Net and **7,894,783** for Attention U-Net. The difference, **132,252 parameters (1.7%)**, is exactly the four attention gates. That makes this a clean comparison, because nothing else changes.

Training setup

Dice + cross-entropy loss; Adam with learning rate 10^{-3} and weight decay 10^{-4}; cosine annealing over 40 epochs; batch size 8; mixed precision; augmentation by flips and 90° rotations with identical parameters for image and mask; model chosen by highest mean foreground Dice on validation; the same seed (42) for both runs so both see the same split.

Results

Model	Parameters	fg Dice	Liver Dice	Tumour Dice	Time
U-Net (baseline)	7,762,531	0.8222	0.9101	**0.7343**	93 min
Attention U-Net	7,894,783	**0.8247**	**0.9172**	0.7322	112 min

Table 4: Final test results. Both models trained 40 epochs on the same split with the same seed.

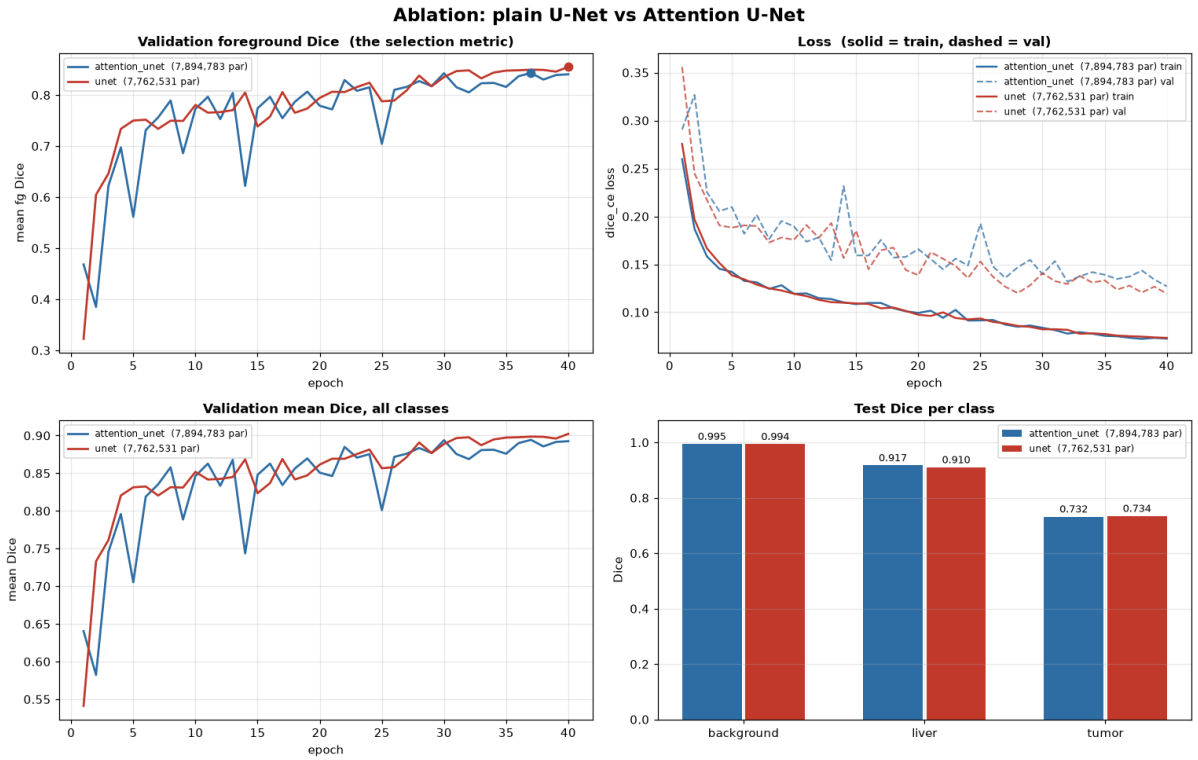


Figure 4: Comparison of the two models. Top left is the metric used to select the best epoch. The loss curves lie almost on top of each other and the per-class bars show how small the differences are.

The difference is not meaningful. Attention U-Net is 0.0025 higher on foreground Dice and 0.0071 higher on liver, but 0.0021 *lower* on tumour. The differences point in different directions depending on the metric, which is what noise looks like. With one run per architecture, anything below about 0.02 cannot be separated from run-to-run variation. Attention U-Net also took 20% longer to train.

Where the real difficulty is. The interesting numbers are not the comparison but the tumour class. For Attention U-Net, tumour precision is 0.816 but recall is only **0.664**, so when it says tumour it is usually right, but it **misses a third of the actual disease**. Of the 582 test slices containing a real lesion, **154 (26%)** scored a Dice of exactly zero, meaning the lesion was missed completely. The model is cautious, which for a screening tool is the wrong direction – you would rather over-flag and let a radiologist dismiss it.

Why accuracy would have hidden all of this. A model predicting background everywhere scores 93.4% pixel accuracy on this test set. My trained model scores 98.9%. That looks like a small gain, but the foreground Dice went from 0.000 to 0.825. Same two models, two metrics, completely different stories.

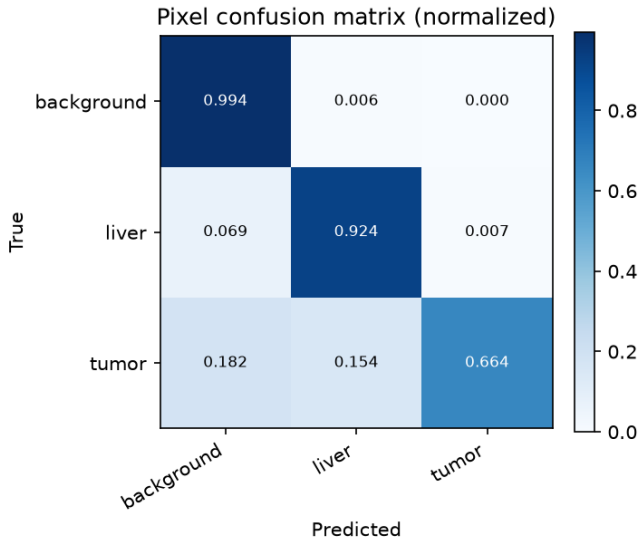


Figure 5: Normalized confusion matrix, Attention U-Net. Rows are the true class. The tumour row is the important one: 66.4% correct, 15.4% predicted as liver, and 18.2% predicted as background – which means the model missed the liver at that location entirely.

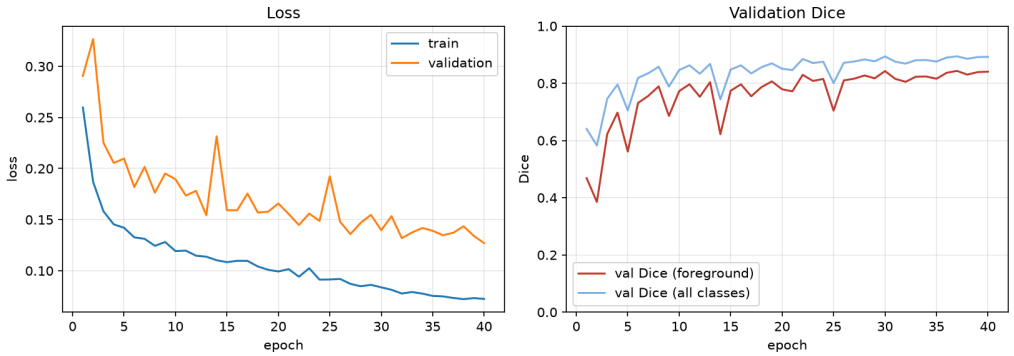


Figure 6: Training curves. Training loss keeps falling while validation loss flattens after about epoch 20, which is mild overfitting. Validation foreground Dice (red, right) is the metric used to choose the best epoch.

Did the attention gates learn anything? Since the numbers showed no improvement, I extracted the α maps from the trained model to check whether the gates were doing anything at all. They were: α was clearly higher inside the liver than outside it, and the maps follow the shape of the organ, getting sharper at higher resolution. So the mechanism works, it just does not help on this task. My explanation is that the liver is large, has good contrast and is always in roughly the same place, so the decoder does not need help finding it – and dropping the 67% of empty slices already solves part of the same problem.

A 3D viewer

The network works on one 2D slice at a time and never sees the slice above or below. Stacking the predictions back in scan order gives a 3D volume, and nothing in the method forces those slices to join up into a sensible shape. To be able to look at that, I wrote a small viewer application in Python. A slider steps through the slices, radio buttons choose the plane and what is drawn on top, and the 3D view beside it is spun by dragging across it.

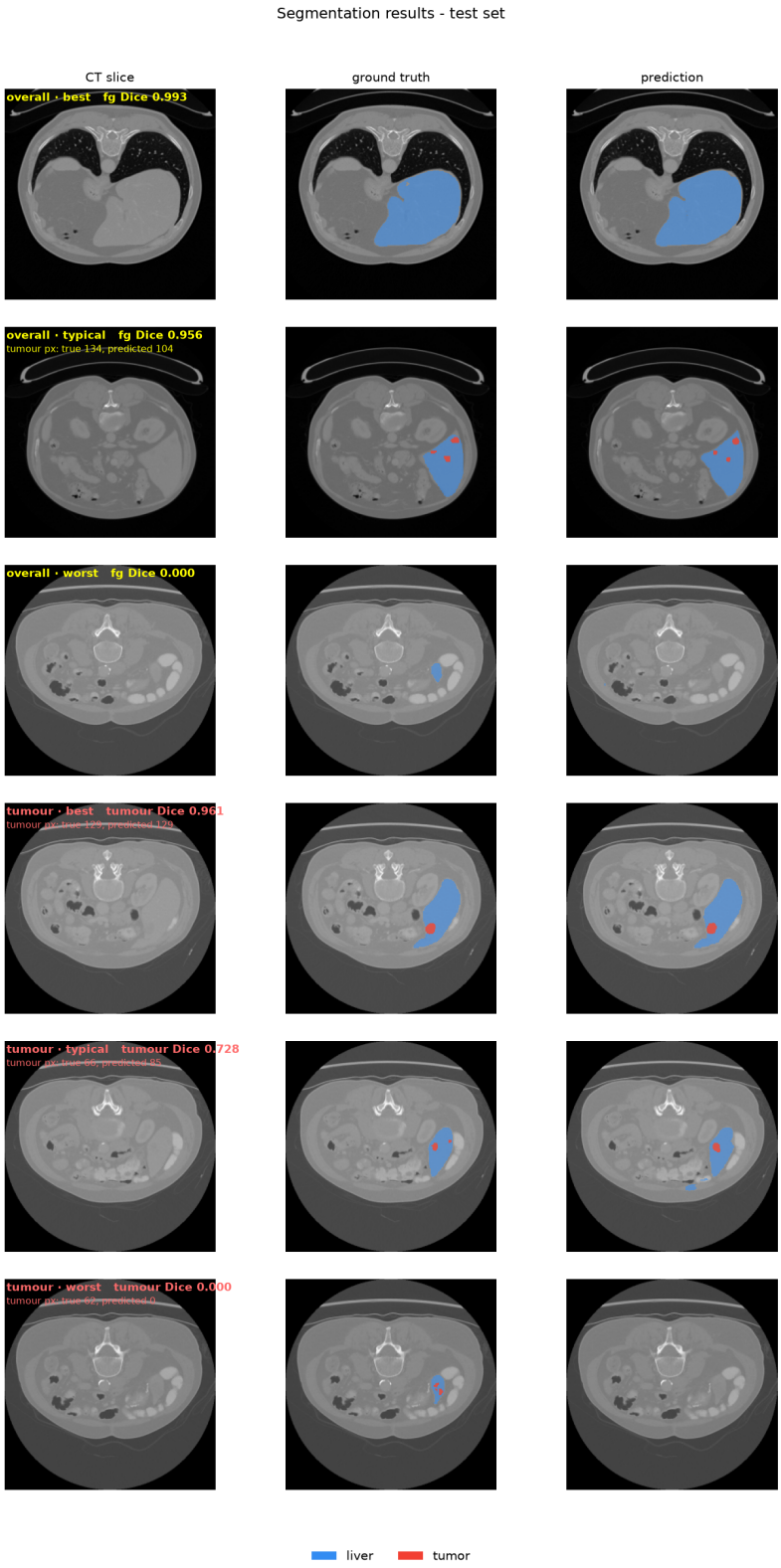


Figure 7: Test predictions, Attention U-Net. Each row shows the CT slice, the ground truth drawn by a radiologist, and the model output. The rows are chosen as best, typical and worst by Dice, and the last rows are picked from slices that contain a real lesion, so the figure shows the failures as well as the successes.

Two things in it turned out to be about speed rather than features, and I only found them by measuring. The first version built the window out of matplotlib, which redraws an entire figure on every change – buttons, sliders and frames included – and hands the whole bitmap to the GUI toolkit, close to two million pixels each time. Rebuilding the window with ordinary `tkinter` widgets means the controls are drawn once and only the image is replaced, about eight times less work per frame. The second is that the 3D is not drawn live at all: rotating a real surface mesh in Python cannot keep up with a mouse, so the rotation is rendered in advance into 36 fixed viewing angles with a depth buffer in `numpy`, and dragging then just picks a frame out of an array.

The viewer loads only the slices whose ground truth contains liver, using the same `has_foreground` rule that training and evaluation use. A CT covers much more of the body than the liver, so most slices of a scan are empty: they contain no true positives at all, so all they can add to a 3D volume is blobs floating where the organ is not. The `.nii.gz` export for 3D Slicer is built the same way. For this patient that is **187 slices out of 689**.

Both the slices and the 3D view can be shown three ways: the ground truth, the prediction, or the **difference**, which colours every voxel by whether the two agree (correct), the model missed it (missed), or it labelled tissue that is not there (false alarm).

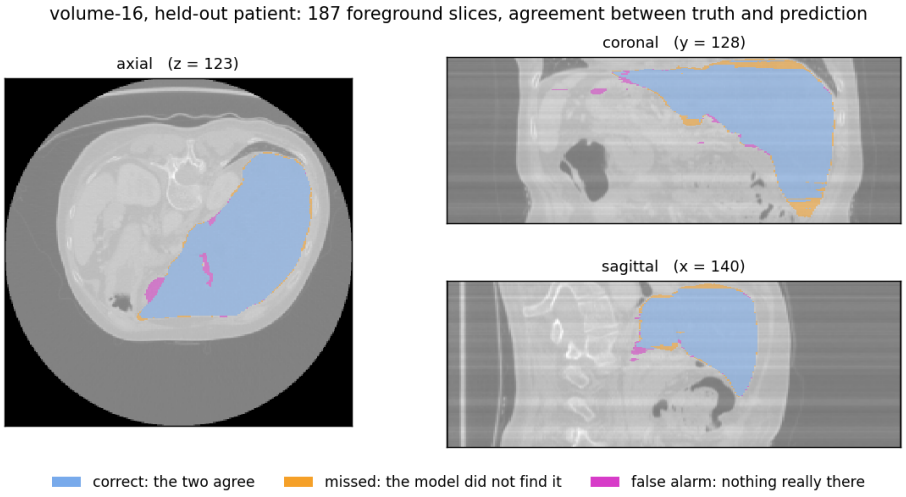


Figure 8: The three planes through one held-out patient. The coronal and sagittal views are reconstructed from the stack of 2D predictions, which is why they are worth looking at: the network never saw them.

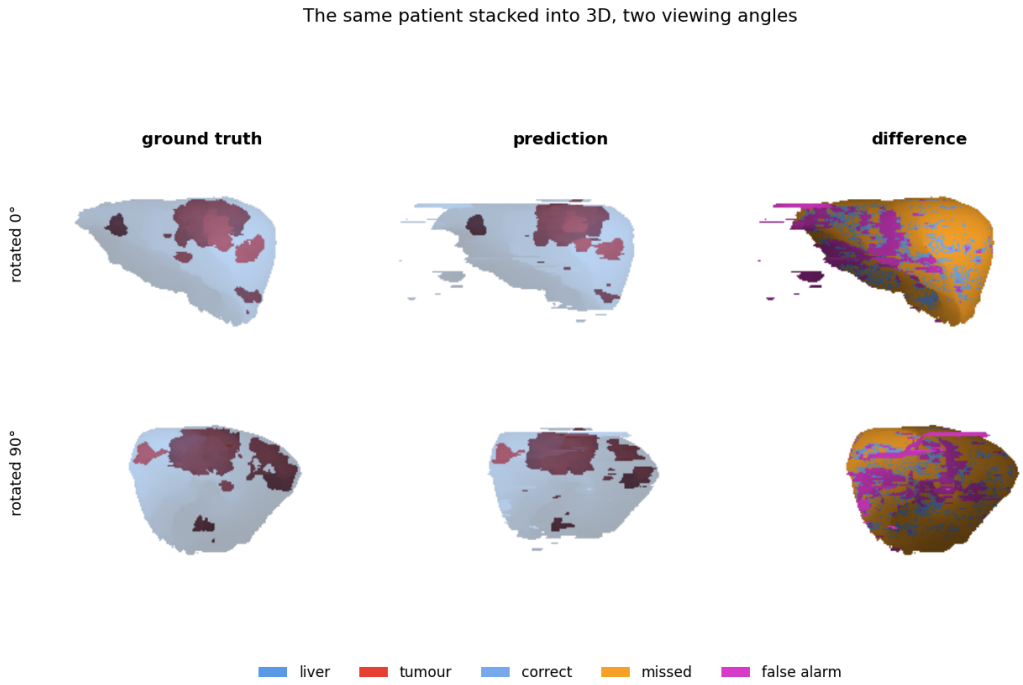


Figure 9: The same patient in 3D at two viewing angles. The predicted organ has the right overall shape, with the errors concentrated at the top and bottom of the liver and a few small blobs detached from it.

Looking at a whole patient at once says things the per-slice numbers do not. For this one:

- correct 812,344 voxels, missed 64,225, false alarm 30,568;
- the model *under*-segments: it misses 2.1 times more than it invents;
- the missed voxels are not only a thin rim around the outline. Only 48.9% of them touch a predicted voxel at all, and 38.7% sit more than two voxels inside the true liver, so there are whole regions it did not find;
- the prediction breaks into **23 connected components** where the liver is one connected organ.

That last point suggests an obvious cleanup: keep only the largest predicted component and throw the rest away. I measured what that would actually do instead of assuming it helps. It removes 9,354 false-positive voxels, 30.6% of them, but it also deletes 1,575 correct ones. So it is worth doing and it is cheap, but it is a smaller win than the picture suggests, because most of the error is in the boundary of the organ rather than in the strays, and that is the harder problem.

Personal contributions

- Wrote the whole codebase with no segmentation framework: configuration, data pipeline, both architectures, loss, metrics, training loop, evaluation and figures.
- Found that the first dataset version had labels damaged by JPEG compression, and moved to a lossless PNG version.
- Found and fixed a data leakage problem by changing from a slice level to a patient level split.
- Chose foreground Dice instead of accuracy or loss for model selection, and showed with a concrete number why accuracy is misleading here.

- Wrote a self test tool and a dataset checking tool that caught several silent bugs, including an incomplete dataset download.
- Generated the architecture documentation automatically from the code, with a check that the diagram, the layer table and the model all agree on the parameter count.
- Wrote an interactive 3D viewer in Python, and used its difference view to find that the predicted liver is 23 disconnected pieces rather than one organ – then measured what the obvious fix would cost instead of assuming it was free.

Limitations

- Only one training run per architecture, so the comparison is not statistically significant.
- Dice is pooled over all test pixels, not averaged per patient. The LiTS benchmark reports per patient scores, so my numbers are not directly comparable to published results.
- The 2D 8-bit export destroys the Hounsfield Unit scale that real volumetric methods use, which limits how good any model on this data can be.
- Mean image brightness varies between patients from 95 to 182, and I use one fixed normalisation for all of them. Per image normalisation would probably help and is the first thing I would try next.

4 Conclusions

The most valuable thing I learned was not how to write a neural network. That part is well documented and the code is fairly short. What surprised me is how much of the work is making sure the numbers mean what you think they mean. Almost every real problem I hit was silent: a dataset with corrupted labels that still loaded fine, a data split that gave good scores for the wrong reason, a metric that looked excellent while the model segmented nothing, a comparison ruined by unequal training budgets, and a half downloaded dataset that produced plausible but wrong results. None of these caused an error message. I only found them because I checked.

That changed how I work. By the end I was writing a small verification tool alongside each feature, and it saved more time than it cost. Computing the same quantity two different ways and comparing them caught real mistakes.

Technically, I now understand convolutional networks, encoder decoder architectures and attention mechanisms well enough to implement them. I also got practical experience I did not have before: setting up CUDA environments on Windows and on Linux, using tmux, and reading GPU telemetry to discover that my bottleneck was data loading rather than computation.

The negative parts were mostly environment problems rather than the work itself. I lost most of a day to a PyTorch installation that broke in a way that looked like a bug in my own code, and the Windows paging file limit blocked features that worked immediately on Linux.

This internship made me more interested in medical image analysis and in computer vision in general. What I find appealing is that the problems are not only about architecture: the data is difficult and imperfect, the metrics need thought, and being wrong has real consequences.